



Striver Recursion Notes

CS Fundamental (Assam Kaziranga University)

Recursion

by

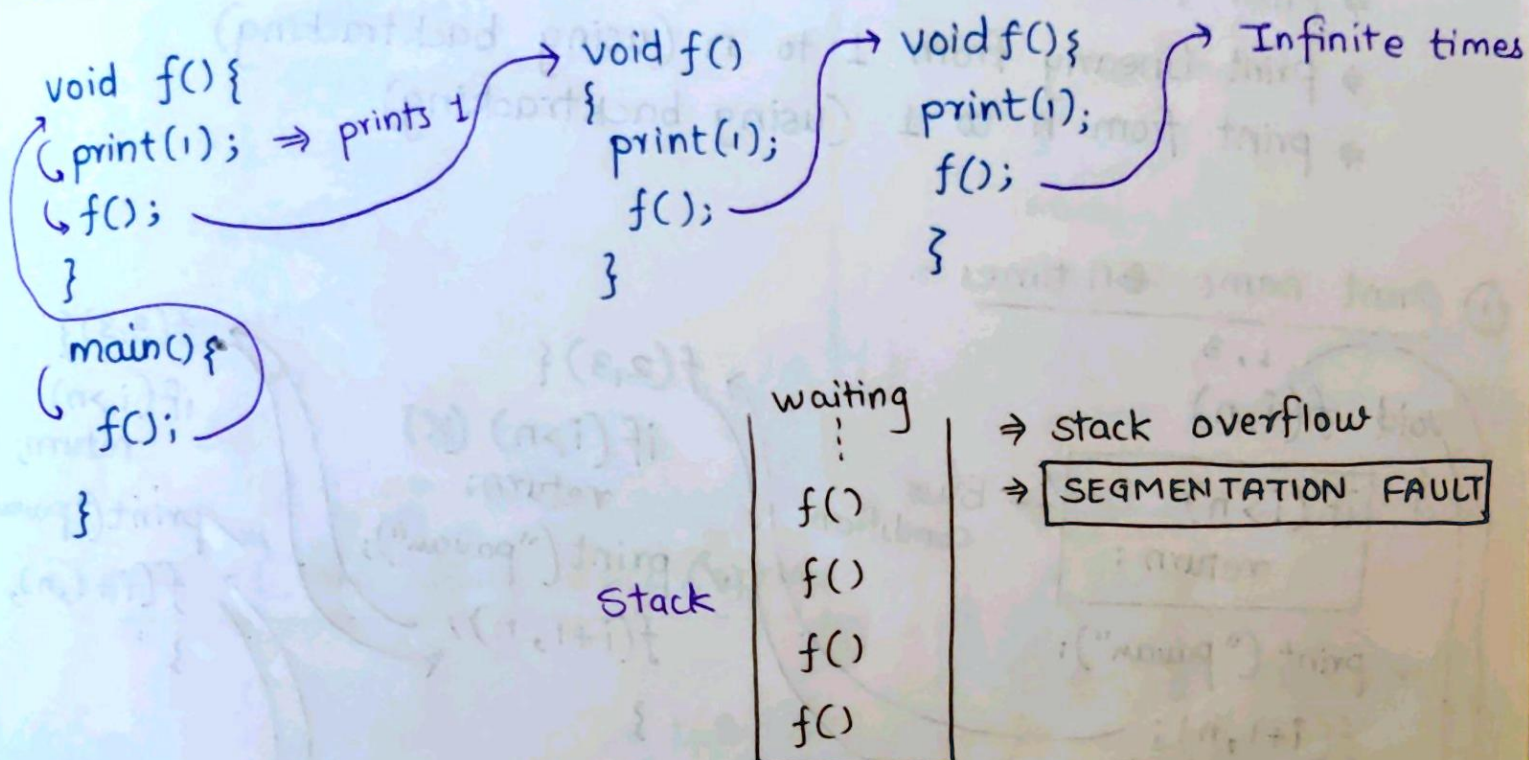
Striver

Recursion :-

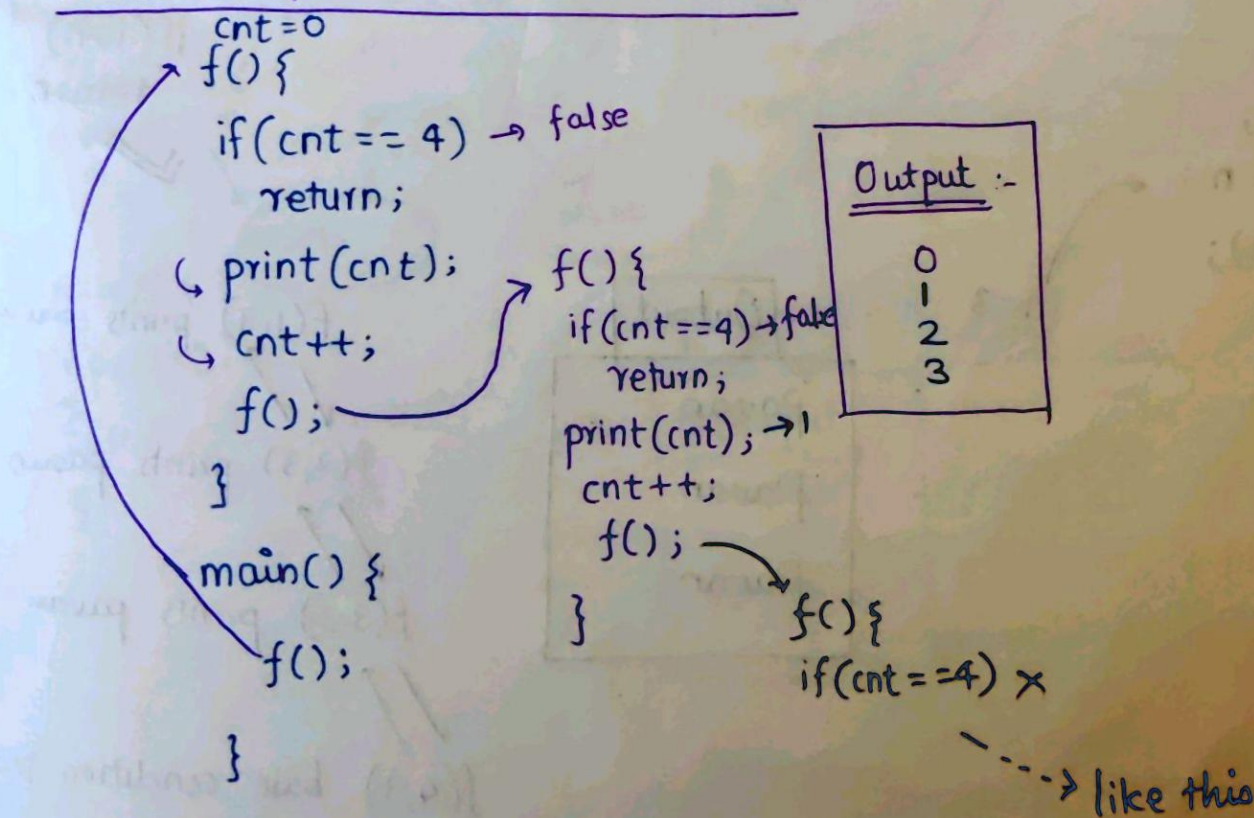
is met.

When a function calls itself until a specified condition

Example :-



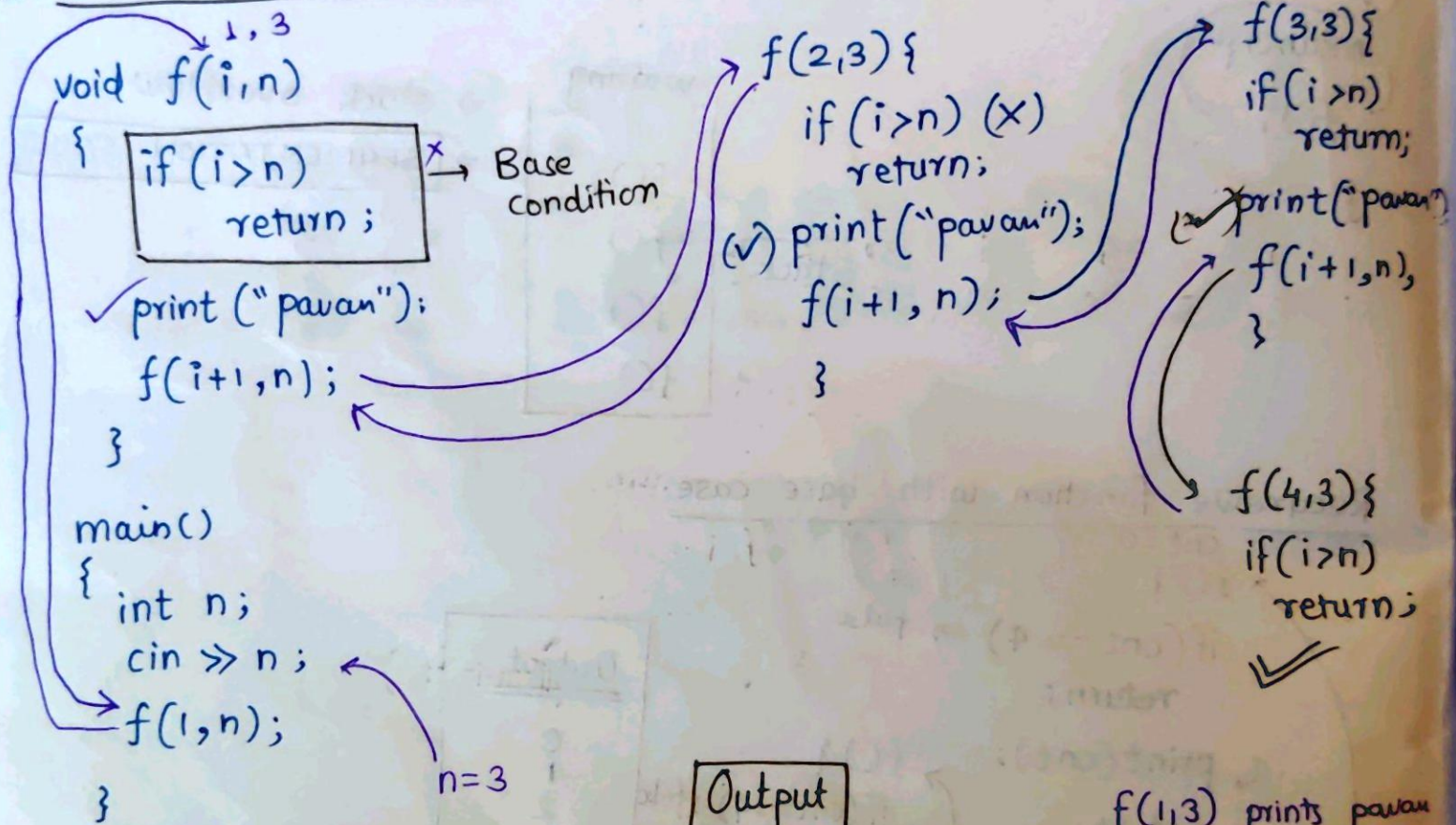
Recursive function with base case:-



Basic Recursion Problems:

- ⇒ Print Name 5 times
- ⇒ Print linearly from 1 to n.
- ⇒ Print from n to 1.
- ⇒ print linearly from 1 to n (using backtracking)
- ⇒ print from n to 1 (using backtracking)

① Print name n times :



Output

```
Pavan
Pavan
Pavan
```

Flow of recursive calls:

- `f(1, 3)` prints pavan
- `f(2, 3)` prints pavan
- `f(3, 3)` prints pavan
- `f(4, 3)` base condition true return;

② print 1 to n :-

```
f(i,n)
{
    if (i > n)
        return;
    print(i);
    f(i+1,n);
}
```

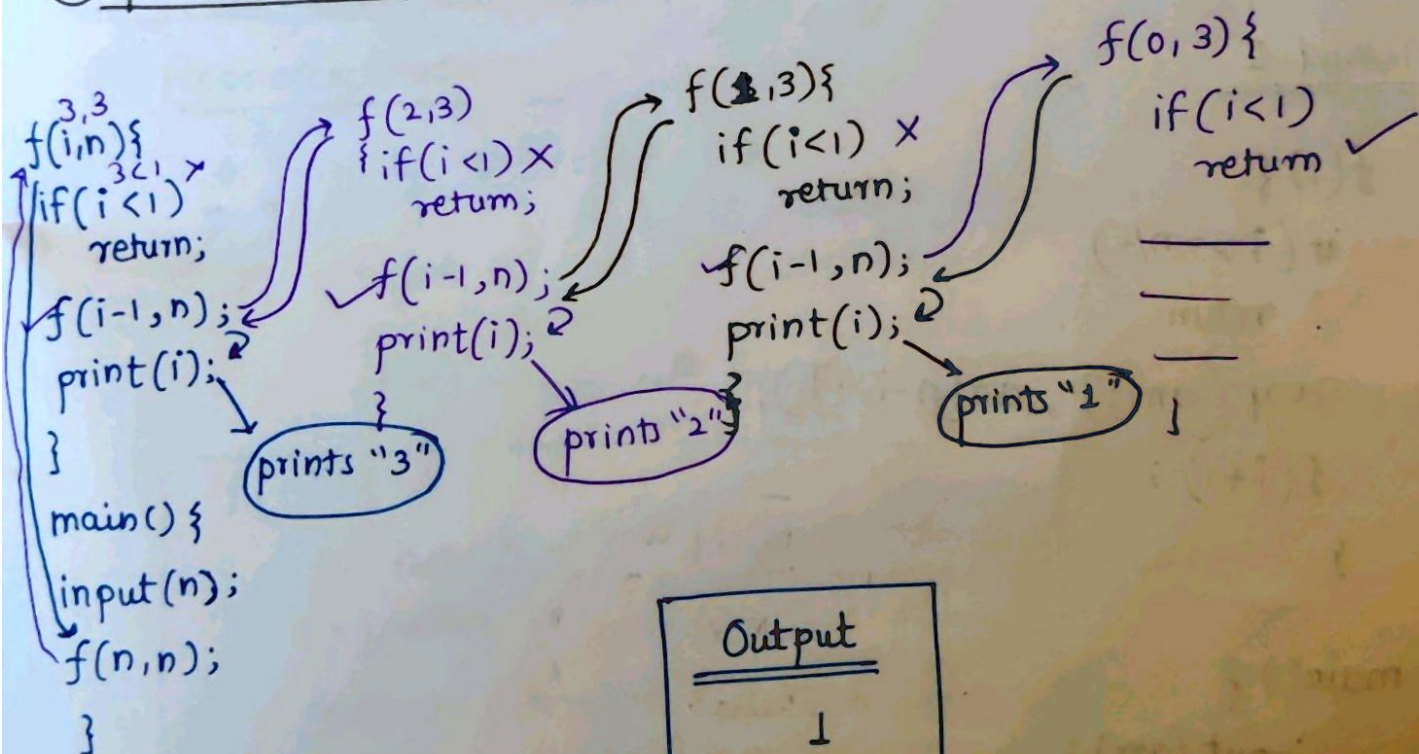
```
main() {
    input(n);
    f(1,n);
}
```

③ print n to 1

```
f(i,n)
{
    if (i < 1)
        return;
    print(i);
    f(i-1,n);
}
```

```
main() {
    input(n);
    f(n,n);
}
```

④ print 1 to n using backtracking :-



Reverse the array using recursion

① Method - I

```
f(l, r) {  
    if (l >= r) return;  
  
    swap(arr[l], arr[r]);  
    f(l+1, r-1);  
}
```

```
main() {  
    input(arr);  
    f(0, n-1);  
}
```

1	2	4	5	3
---	---	---	---	---

→ It will swap until $(l \geq r)$ this condition ~~is~~ will get true.

② Method - II

```
f(i) {  
    if (i >= n/2)  
        return;  
  
    swap(arr[i], arr[n-i-1]);  
    f(i+1);  
}
```

```
main() {  
    input(arr);  
    f(0);  
}
```


check if a string is palindrome

"MADAM" $\xrightarrow{\text{reverse}}$ "MADAM" $\Rightarrow$ This is palindrome

```
f(i) {  
    if (i >= n/2)  
        return true;  
  
    if (s[i] != s[n-i-1])  
        return false;  
  
    return f(i+1);  
}
```

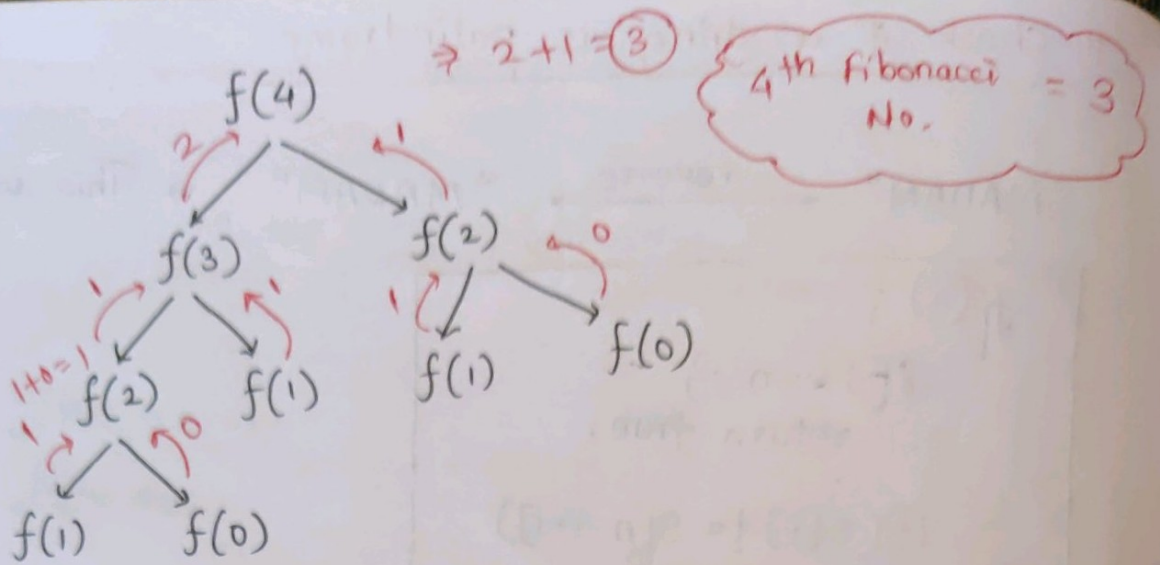
Multiple Recursion calls

① Fibonacci Number

0 1 1 2 3 5 8 13
↑ ↑ ↑ ↑
0th 1st 2nd 3rd

$f(N) \longrightarrow N^{\text{th}}$ Fibonacci Number

```
f(n) {  
    if (n <= 1)  
        return n;  
  
    return f(n-1) + f(n-2);  
}
```

Print all Subsequence:-

A contiguous / non-contiguous sequence which follows the order

arr = [3, 1, 2]

[]
 [3]
 [1]
 [2]
 [3, 1]
 [1, 2]
 [3, 2]
 [3, 1, 2]

#. Subsequence = 8

#. Subsequence = 2^n

arr =

3	1	2
---	---	---

Take / Non-take

↓
on indices

for [3, 2] Take X Take
 i ⇒ 0 1 2

[1, 2] X ✓ ✓

[3, 1] ✓ ✓ X

[3, 1, 2] ✓ ✓ ✓

{ } X X X

✓. Imp

arr

3	1	2
0	1	2

f(ind, [])

```
{ if (ind >= n)
  print([]);
  return;
```

```
  [].push-back(arr[i]);
```

```
  f(ind+1, []);    → Take
```

```
  [].pop-backremove(arr[i]);
```

```
  f(ind+1, []);    → Not-take
```

```
}
```

```
main() {
```

```
  arr = {3, 1, 2}
```

```
  f(0, []);
```

```
}
```



```
f(ind, []) {
    if (ind >= n)
        print([]);
    return;
```

```
[3] [].push_back(arr[i]);  
    f(ind+1, []); → Tak  
    [].remove(arr[i]);  
    f(ind+1, []);  
}
```

```

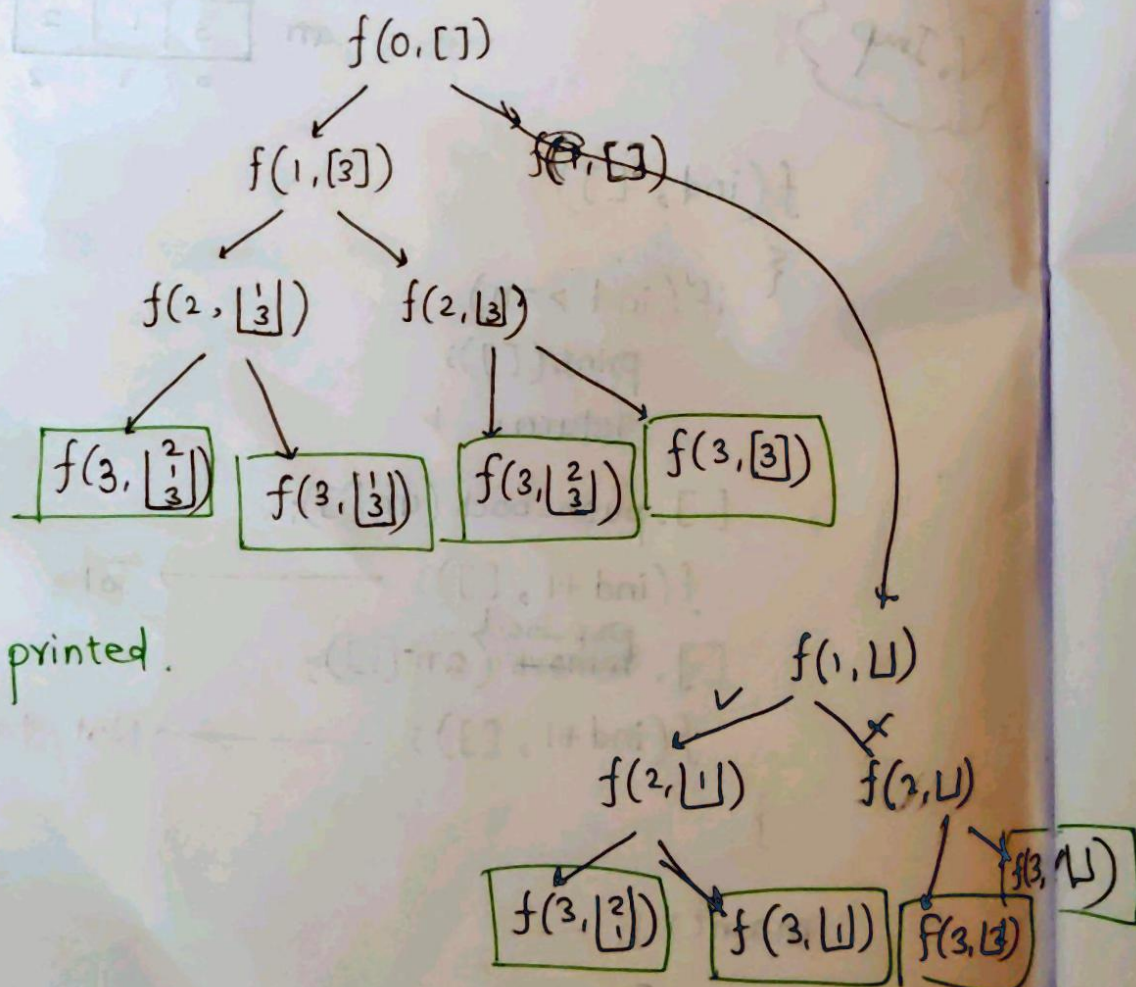
→ f(1, [3])
    { if() ✗
[3, 1] [3]. pop(arr[i]),
      f(2, [3, 1]); ✓
[3, 1]. remove(arr[

```

$\rightarrow f(2, [3, 1])$
 $\{ \text{if } () \}$
 $[3, 1].\text{pb}(\text{arr}[2]) \Rightarrow [3, 1, 2]$
 $f(3, [3, 1, 2]);$
 $); [3, 1, 2].\text{remove}(\text{arr}[2]);$
 $\downarrow f(3, [3, 1]);$
 $\Rightarrow [3, 1]$
 $f(3, [3, 1, 2])$
 base condition ✓
 $f(3, [3, 1]);$
 base Condition ✓

In this way the whole code is working

Output

$$[3, 1, 2]$$
 $[3, 1]$ 

all Printing Subsequences whose sum is k

arr = [1, 2, 1]

k = 2

ind ds sum
f(0, [], 0)

f(1, [1], 0)

f(1, [1], 1)

f(2, [1, 2], 2)

f(2, [1], 0)

f(2, [1, 2], 3)

f(2, [1], 1)

~~f(3, [1, 2], 3)~~

f(3, [1, 2], 2)
print

~~f(3, [1], 1)~~

~~f(3, [1], 0)~~

~~f(3, [1, 2], 4)~~

~~f(3, [1, 2], 3)~~

f(3, [1], 2)
Sum == k
print

~~f(3, [1], 1)~~

Sum != k
(don't print)

f(ind, ds, sum)

```
{
    if (i == n)
        if (sum == k)
            print(ds);
        return;
```

```
    ds.push-back(arr[ind]);
    sum += arr[ind];
```

```
    f(ind+1, ds, sum);
```

```
    ds.pop-back();
    sum -= arr[ind];
```

```
    f(ind+1, ds, sum);
```

```
}
```


- Print any subsequence whose sum is k.

Technique to print only one answer

bool f() {

base condition

condition $\rightarrow$ satisfied
return true

condition $\rightarrow$ not satisfied
return false

if (f() == true)

return;

f();

return false

}

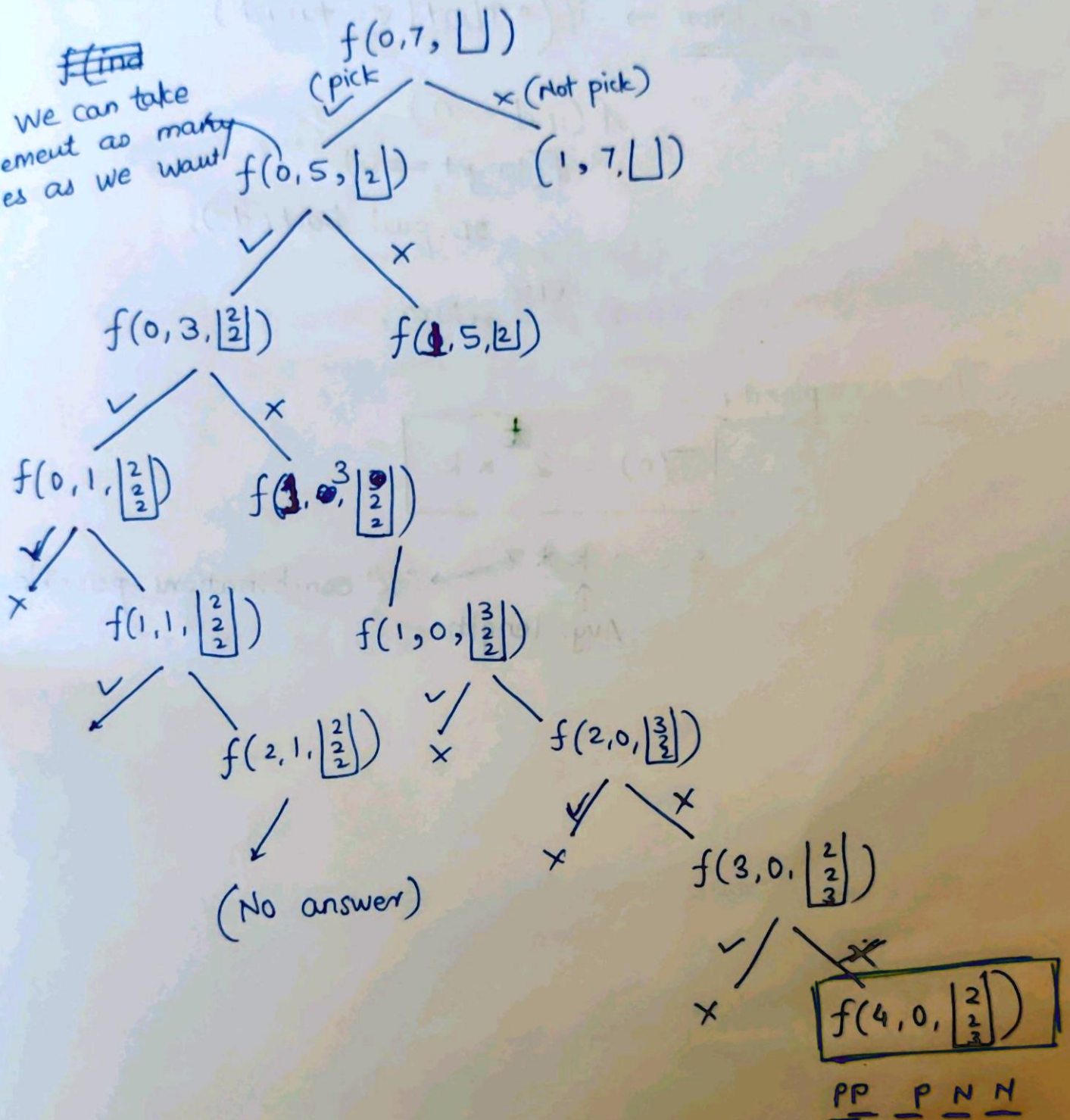
Combination Sum (Leetcode)

arr = [2, 3, 6, 7] target = 7

Output $\Rightarrow$ [[2, 2, 3], [7]]

~~find~~

We can take element as many times as we want



$f(\text{ind}, \text{target}, \text{ds})$
 $f(\text{ind}, \text{target} - a[\text{ind}], \text{ds})$ $f(\text{ind} + 1, \text{target}, \text{ds})$

Condition $\rightarrow$ if ($a[\text{ind}] \leq \text{target}$)

if ($\text{ind} == n$)

if ($\text{target} == 0$)

2D.push_back(ds);

else return;

Time Complexity :-

$$T(n) = 2^k \times k$$

$S.C. = k * x$
 $\uparrow$ $\rightarrow$ 'x' combinations possible
 Avg. length

Code:-

```
findCombination(int ind, target, arr, vector<vector<int>>&ans, ds)
{
    if (ind == arr.size())
    {
        if (target == 0)
        {
            ans.push_back(ds);
        }
        return;
    }
    // pick the element
    if (arr[ind] <= target)
    {
        ds.push_back(arr[ind]);
        findCombination(ind, target - arr[ind], arr, ans, ds);
        ds.pop_back();
    }
    // not pick
    findCombination(ind+1, target, arr, ans, ds);
}
```

→ When upper recursion is totally executed and we have not got our answer so we have to empty our (pop-back()) that element ds.

• Combination Sum - II (Leetcode)

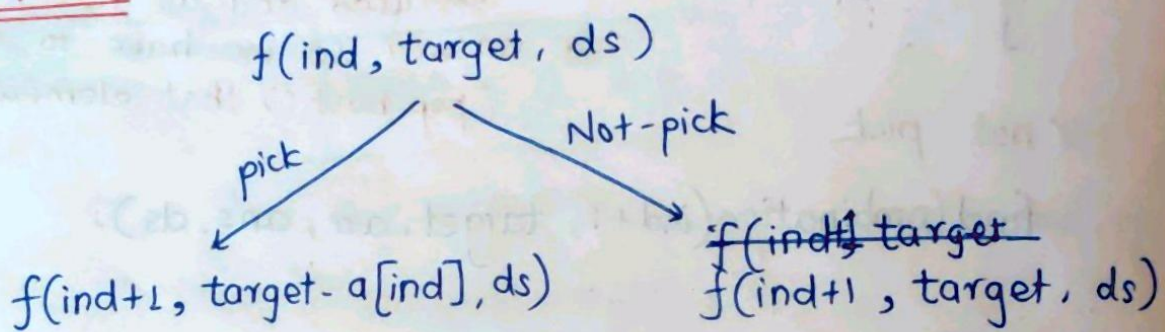
arr[] and target will be given to us.
Find all unique combinations in "arr[]" where array elements sums to target.

- ⇒ Each number can be taken only once.
- ⇒ The solution set must not contain duplicate combinations.

arr[] = [1, 1, 1, 2, 2] target = 4

Combinations → [[1, 1, 2],
 [2, 2]]

Brute Force :-



⇒ If we pick the element then we have to move the "ind" by 1 cause in this problem we can take ~~any~~ element only once.

```
findCombination(ind, target, arr, set <vector <int>> &ans, ds)
{
    if (ind == arr.size()) {
        if (target == 0) {
            ans.insert(ds);
        }
        return;
    }
    if (arr[ind] <= target) {
        ds.push-back(arr[ind]);
        findCombination(ind+1, target - arr[ind], arr, ans, ds);
        ds.pop-back();
    }
    findCombination(ind+1, target, arr, ans, ds);
}
```


modified approach :-

Instead of pick and non-pick we will pick subsequences.

arr[] = [1, 1, 1, 2, 2]
0 1 2 3 4

target = 4

first sort the array

f(0, 4, [])

pick 0th index

pick 1st

pick 2nd

3rd

4th

x (Avoid repeated combinations)

f(1, 3, [1])

1st

2nd

3rd

4th

(We will not perform these recursion cause we have started already with "1"
To avoid repeated combinations)

f(4, 2, [2])

(here we have to move the index by 1 cause we need sorted combinations and array was sorted so if we take "2" into the ds then we can't take elements from left part of array of "2".
So ind + 1 $\Rightarrow$ 3 + 1 $\Rightarrow$ 4)

f(2, 2, [1])

(Avoid same Combo.)
X (Not pick)

f(4, 1, [2])

X (2 has been picked already)

2nd

3rd

4th

f(3, 1, [1])

f(4, 0, [2])

X (Avoid)

Here we have got one combination

3rd

4th

X

(target $\leq$ arr[ind])

(target == 0)

combinations

[1, 1, 2],
[2, 2]

f(4, 2, [2])

4th

f(5, 0, [2])

Another answer

f(ind, target, ds, ans)

if(arr[i] > target) $\rightarrow$ break

looping from (ind to n-1) $\Rightarrow$ (i)

possibility

No possibility

ds.pb(arr[i])

f(ind + 1, target - arr[i], ds, ans)

ds.pop-back()

T.C. = $2^n \times k$

S.C. = $k \times n$

Code

```
void findCombination(ind, target, arr, vector<vector<int>> &ans, ds) {  
    if (target == 0) {  
        ans.push_back(ds);  
        return;  
    }  
    for (int i = ind; i < arr.size(); i++) {  
        if (i > ind && arr[i] == arr[i-1])  
            continue;  
        if (arr[i] > target)  
            break;  
        ds.push_back(arr[i]);  
        findCombination(i+1, target-arr[i], arr, ans, ds);  
        ds.pop_back();  
    }  
}
```

```
vector<vector<int>> combinationSum2(vector<int> &candidates, target)  
{  
    sort(candidates.begin(), candidates.end());  
    vector<vector<int>> ans;  
    vector<int> ds;  
    findCombination(0, target, candidates, ans, ds);  
    return ans;  
}
```


• Subset - Sum - I •

Given an array . print sums of all subsets in it.
Output should be printed in increasing order of sums.

Ex $[3, 1, 2]$ $n=3$

	Sum
$\{\}$	$\Rightarrow 0$
$\{3\}$	$\Rightarrow 3$
$\{1\}$	$\Rightarrow 1$
$\{2\}$	$\Rightarrow 2$
$\{3, 1\}$	$\Rightarrow 4$
$\{1, 2\}$	$\Rightarrow 3$
$\{3, 2\}$	$\Rightarrow 5$
$\{3, 1, 2\}$	$\Rightarrow 6$

o/p

$0, 1, 2, 3, 3, 4, 5, 6$

for any 'n'

The no. of subsets = 2^n

Brute force

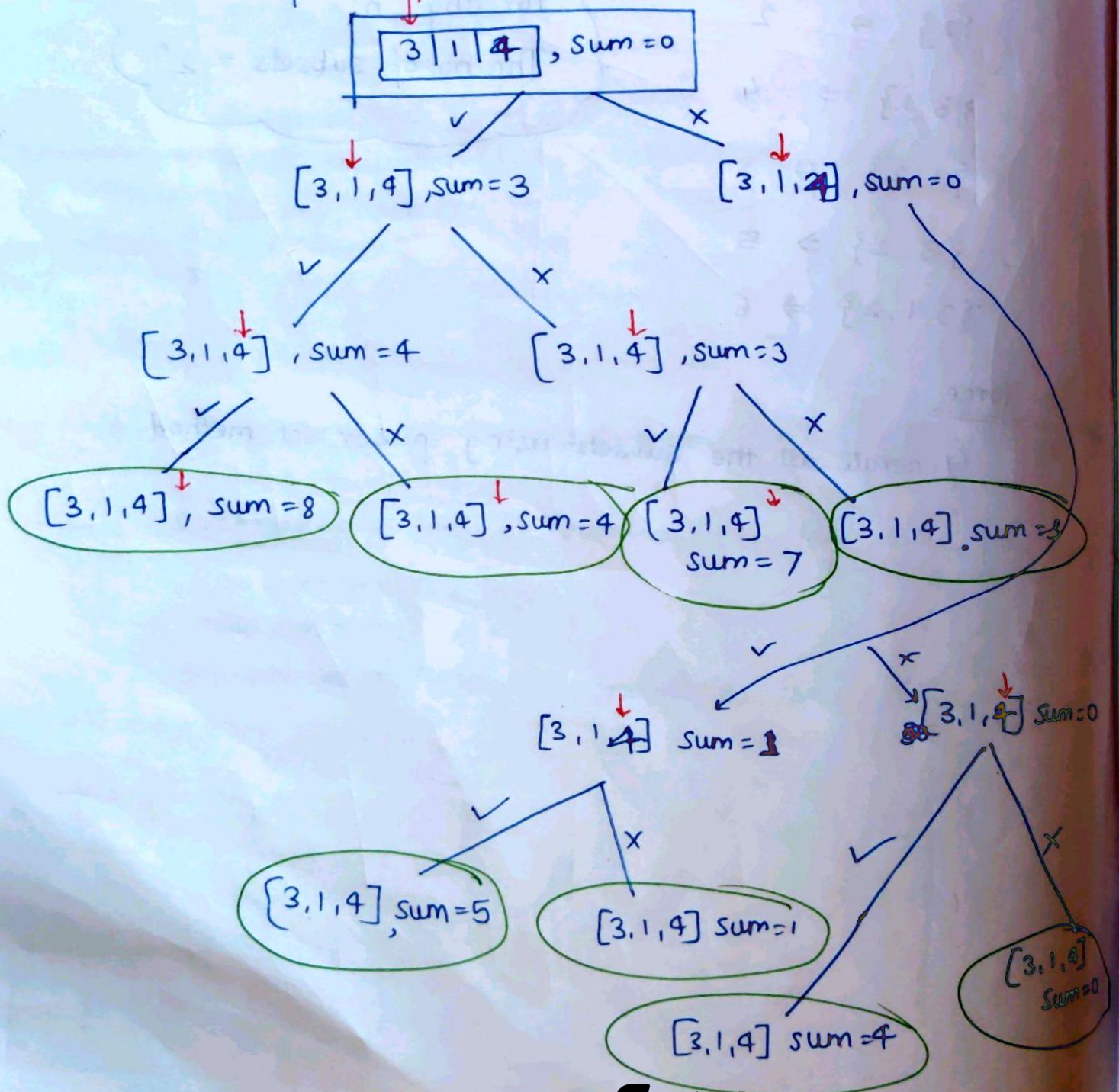
Generate all the subsets using power set method

[3, 1, 4]

$$\frac{\checkmark}{0} \frac{\times}{1} \frac{\checkmark}{2} \Rightarrow [3, 4]$$

$$\frac{\checkmark}{0} \frac{\checkmark}{1} \frac{\times}{2} \Rightarrow [3, 1]$$

Select / Not-select
pointer



[8, 4, 7, 3, 5, 1, 4, 0]

Sort

[0, 1, 3, 4, 4, 5, 7, 8]

Recursive Calls

$f(\text{ind}, \text{sum})$

$f(\text{ind}+1, \text{sum} + \text{arr}[\text{ind}])$

$f(\text{ind}+1, \text{sum})$

base case

if ($\text{ind} == n$)

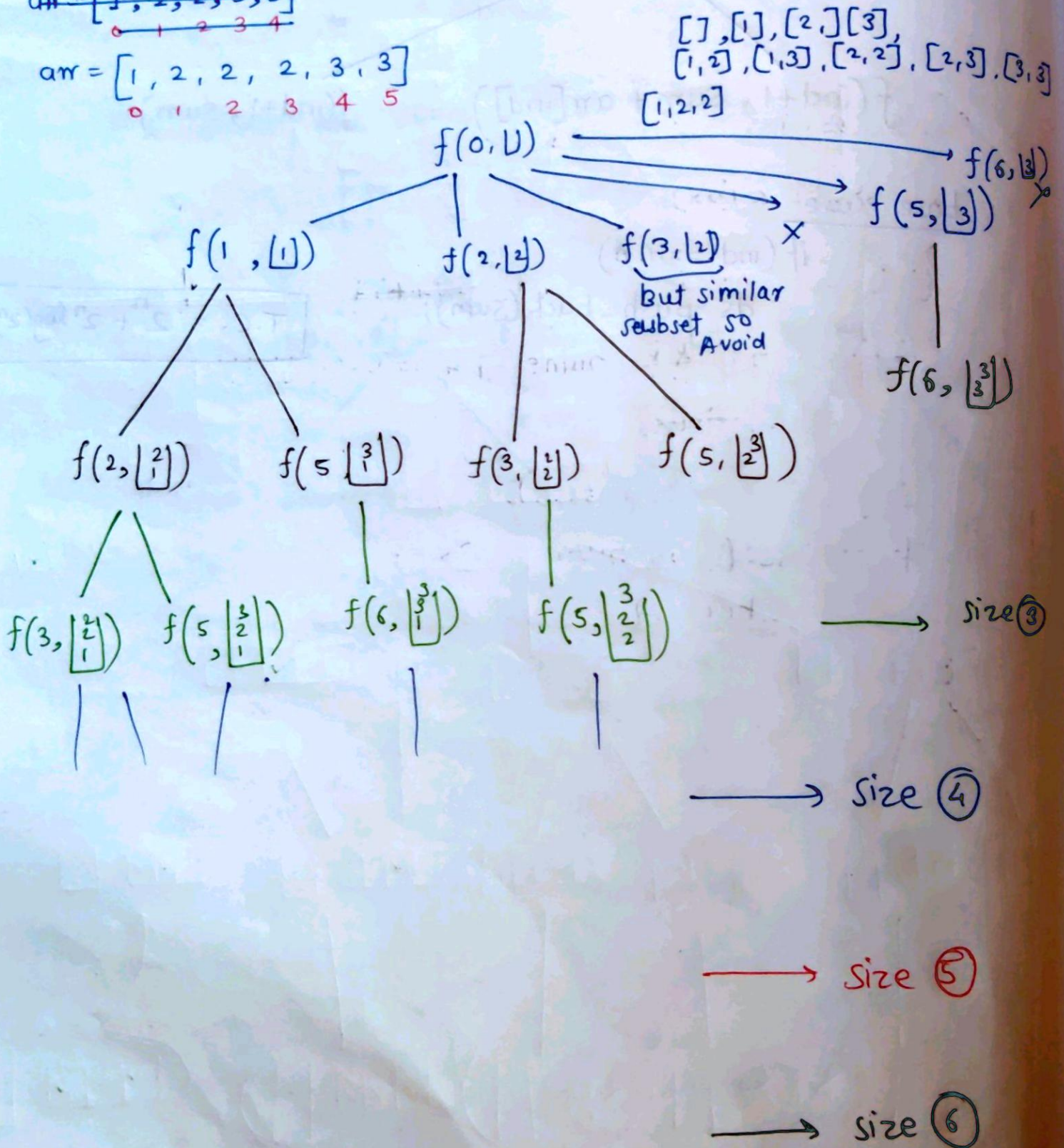
ds.push_back(sum);

T.C. = $2^n + 2^n \log(2^n)$

Subset Sum - II

Given an integer array `nums` that may contains duplicates, return all possible subsets.

The solution set must not contain duplicates

$$\text{arr} = [1, 2, 2, 3, 3]$$
$$\text{arr} = [1, 2, 2, 2, 3, 3]$$


$f(ind, ds) \rightarrow \text{size} \Rightarrow s$
 $ds.pb(arr[i]); \Rightarrow \text{ind to } n-1$
 $f(i+1, ds) \rightarrow (s+1) \text{ size}$

Code

```

void findSubsets(ind, nums, ds, ans)
ans.push_back(ds);

for(int i = ind ; i < nums.size() ; i++) {
    if (i != ind && nums[i] == nums[i-1])
        continue;

    ds.push_back(nums[i]);
    findSubsets(i+1, nums, ds, ans);
    ds.pop_back();
}
    
```


● Print all permutations of a string/array :-

Given an array "nums" of distinct integers, return all possible permutations.

Ex:- nums = [1, 2, 3]

output $\Rightarrow$ [1, 2, 3],
[1, 3, 2],
[2, 1, 3],
[2, 3, 1],
[3, 1, 2],
[3, 2, 1]

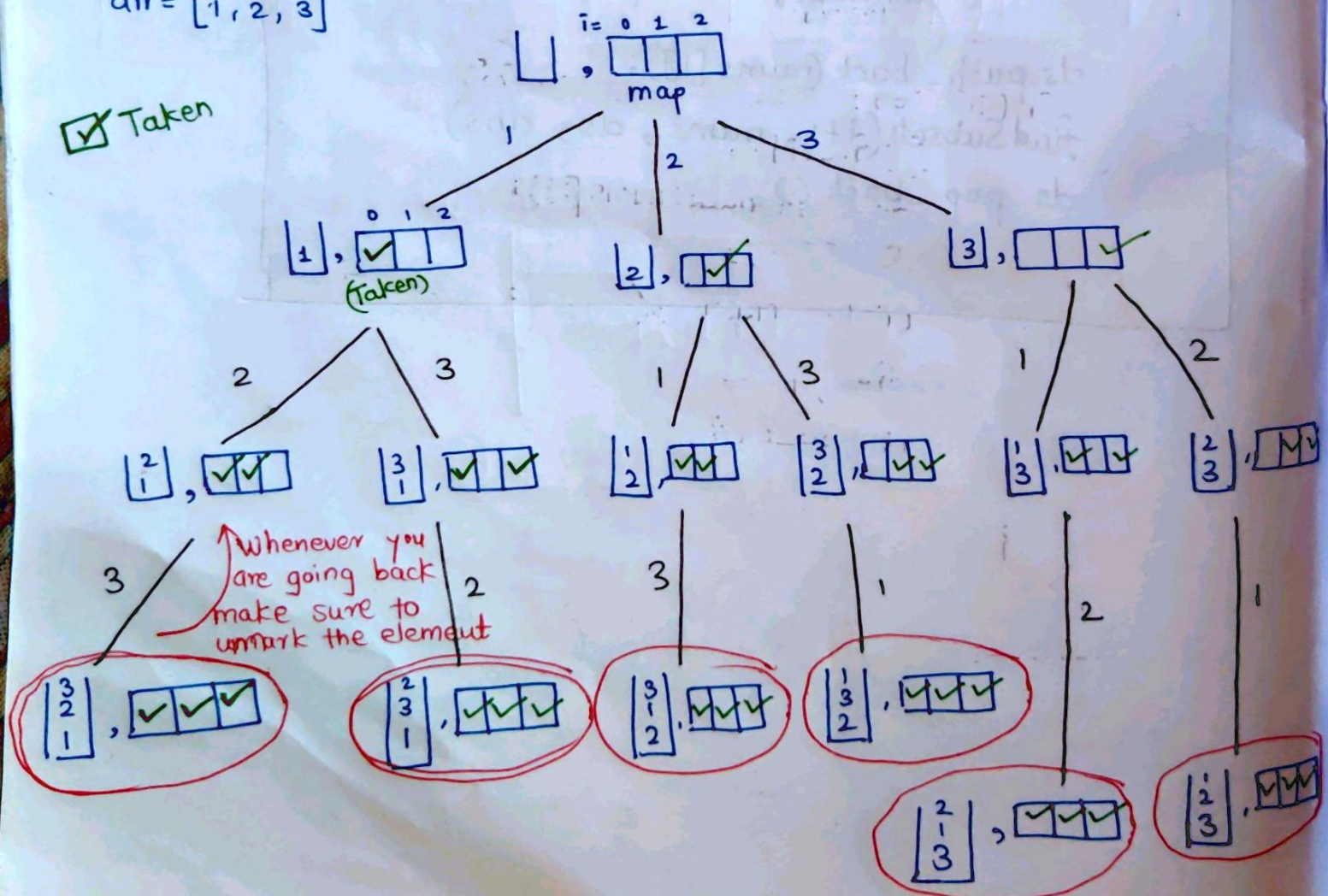
① Using Extra Space Complexity :-

If n elements is given then

No. of possible permutations = $n!$

arr = [1, 2, 3]

✓ Taken



$f(ds, \text{hashmap})$
 $ds.\text{push_back}(a[i])$
 $\text{map}[i] = 1$ (Mark it as taken)
 loop $(0 - n-1)$
 if $(i \text{ is not in map})$

$f(ds, \text{map})$

if $(ds.\text{size}() == n)$
 $\text{ans.pb}(ds);$

Time Complexity = $O(n! \times n)$
 Space Complexity = $O(n) + O(n)$

```

printPermutation(nums, ds, ans, freq) {
    if (ds.size() == nums.size())
        ans.push_back(ds);
        return;

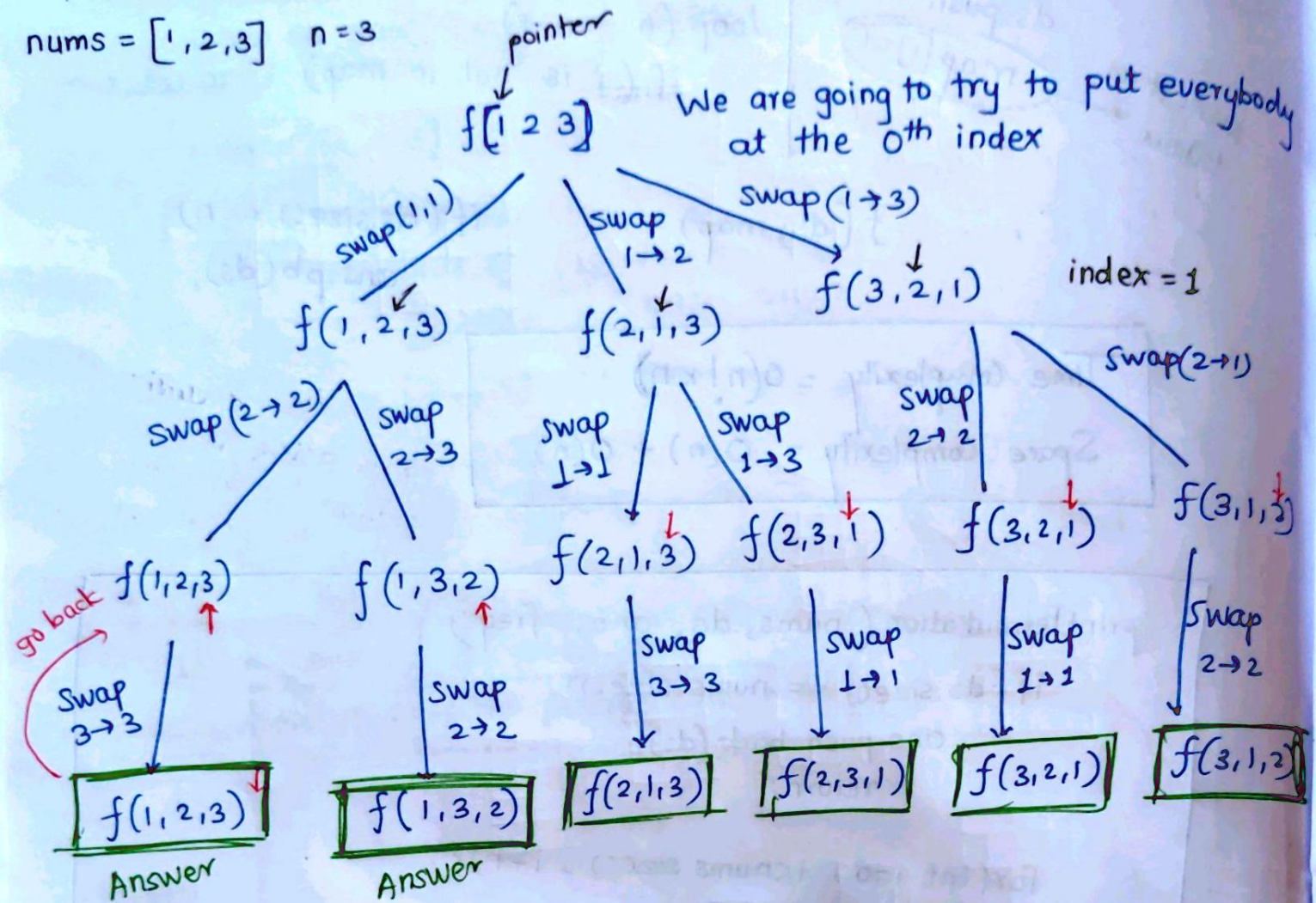
    for (int i = 0; i < nums.size(); i++) {
        if (!freq[i]) {
            ds.push_back(nums[i]);
            freq[i] = 1;  $\rightarrow$  marked as picked
            printPermutation(nums, ds, ans, freq);
            freq[i] = 0;  $\rightarrow$  unmark the position in freq
            ds.pop_back();  $\rightarrow$  Remove element from (ds);
        }
    }
}
    
```

```

main() {
    takeInput(nums);
    vector<int> ds;
    vector<vector<int>> ans;
    for vector int freq[nums.size()];
    for (int i = 0; i < nums.size(); i++)
        freq[i] = 0;
    }
    
```


② Optimized Approach :-

nums = [1, 2, 3] n = 3



$f(\text{ind}, \text{arr})$

$i \rightarrow (\text{ind} \rightarrow n-1)$

$\text{swap}(a[\text{ind}], a[i]);$

$f(\text{ind}+1, \text{arr})$

Base Case :-

if (ind == n)

ans.push_back(arr);

return;

N-Queens

```
f(ind, nums, ans) {
```

```
    if (ind == nums.size())  
        ans.push_back(nums);  
    return;
```

```
    for (int i = ind; i < nums.size(); i++) {  
        swap(nums[ind], nums[i]);
```

```
        f(ind+1, nums, ans);
```

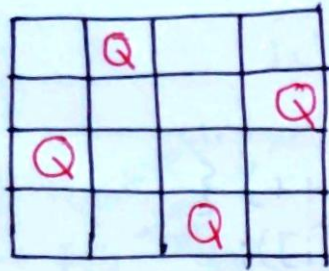
```
        swap(nums[ind], nums[i]);
```

```
    }
```

→ After recursive call
when going backwards
we have to reswap
the elements.

N-Queens

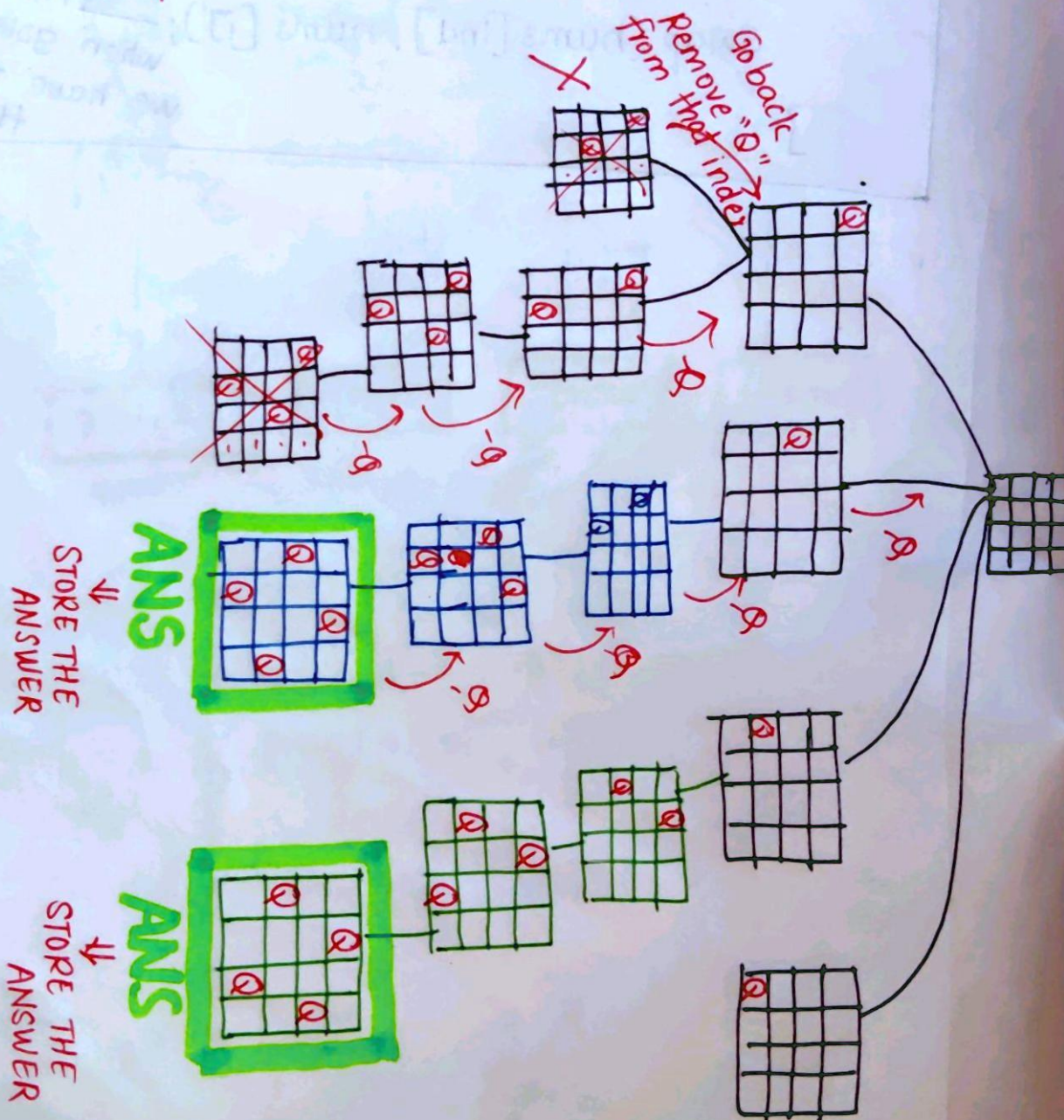
n=4

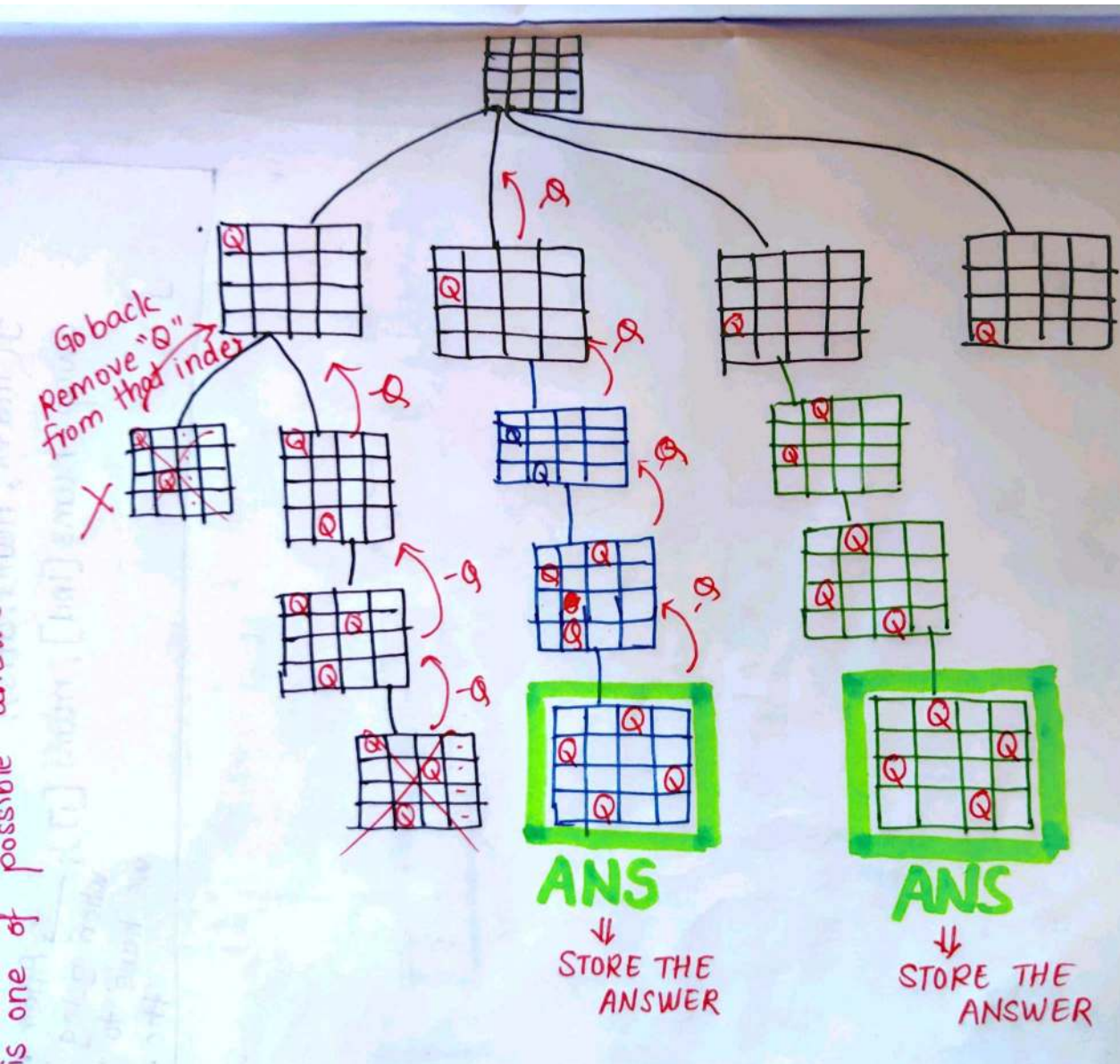


Rules

- ① Every row should have a queen.
- ② Every column should have a queen.
- ③ None of the Queen attack each other.

This is one of possible answer.





pseudo Code

$\forall(\text{col}) \{$ → filling each column

```
for (i=0 ; i < row ; i++) {  
    if (fill ✓) {  
        Board[row][col] = 'Q';  
        Solve(col+1);  
        Board[row][col] = '.';  
    }  
}
```

Code :-

```
bool isSafe (int row, int col, vector<string> board, int n) {  
    int duprow = row;  
    int dupcol = col;  
    while (row >= 0 && col >= 0) {  
        if (board[row][col] == 'Q') {  
            return false;  
        }  
        row--;  
        col--;  
    }  
    row = duprow;  
    col = dupcol;  
while (row < n &&  
    while (col >= 0) {  
        if (board[row][col] == 'Q') {  
            return false;  
        }  
        col--;  
    }  
}
```



```

row = duprow;
col = dupcol;
while (row < n && col >= 0) {
    if (board[row][col] == 'Q') {
        return false;
    }
    row++;
    col--;
}
return true;
}

```

```

void solve (int col, vector<string> &board,
            vector<vector<string>> &ans, int n) {
    if (col == n) {
        ans.push_back(board);
        return;
    }
    for (int row = 0; row < n; row++) {
        if (isSafe(row, col, board, n)) {
            board[row][col] = 'Q';
            solve(col + 1, board, ans, n);
            board[row][col] = '.';
        }
    }
}

```



```
vector<vector<string>> solveQueens(int n){
```

```
    vector<vector<string>> ans;
```

```
    vector<string> board(n);
```

```
    string s(n, '.');
```

```
    for (int i=0; i<n; i++){
```

```
        board[i] = s;
```

```
    }
```

```
    solve(0, board, ans, n);
```

```
    return ans;
```

```
}
```


Sudoku Solver

- 1) The digit 1-9 appears only once in a row.
- 2) The digit 1-9 appears only once in a column.
- 3) The digit 1-9 appears only once in 3x3 grid.

Code :-

```
void SolveSudoku(vector<vector<char>>& Board) {  
    Solve(board);  
}  
  
bool Solve(vector<vector<char>>& board) {  
    for(int i=0 ; i< board.size() ; i++) {  
        for(int j=0 ; j< board[0].size() ; j++) {  
            if (board[i][j] == '.') {  
                for(char c='1' ; c<='9' ; c++) {  
                    if (isValid(board, i, j, c)) {  
                        board[i][j] = c ;  
                        if (Solve(board) == true)  
                            return true;  
                        else  
                            return false ; board[i][j] = '.' ;  
                    }  
                }  
                return false ;  
            }  
        }  
    }  
    return true ;  
}
```



```

bool isValid(vector<vector<char>>& board, int row, int col, char c) {
    for(int i=0 ; i<9 ; i++) {
        if(board[i][col] == c)
            return false;
        if(board[row][i] == c)
            return false;
        if(board[3 * (row/3) + (i/3)][3 * (col/3) + (i%3)] == c)
            return false;
    }
    return true;
}

```


• M-Coloring problem

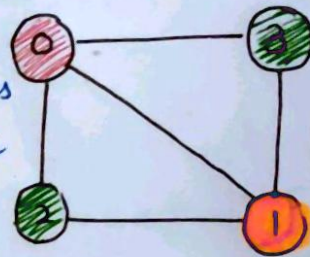
Given an undirected graph and an integer M . The task is to determine if the graph can be colored with at most M colors such that no two adjacent vertices of the graph are colored with the same color. Here coloring of graph means the assignment of colors of all vertices. Print 1 if possible to colour all the vertices and 0 otherwise.

Ex:-

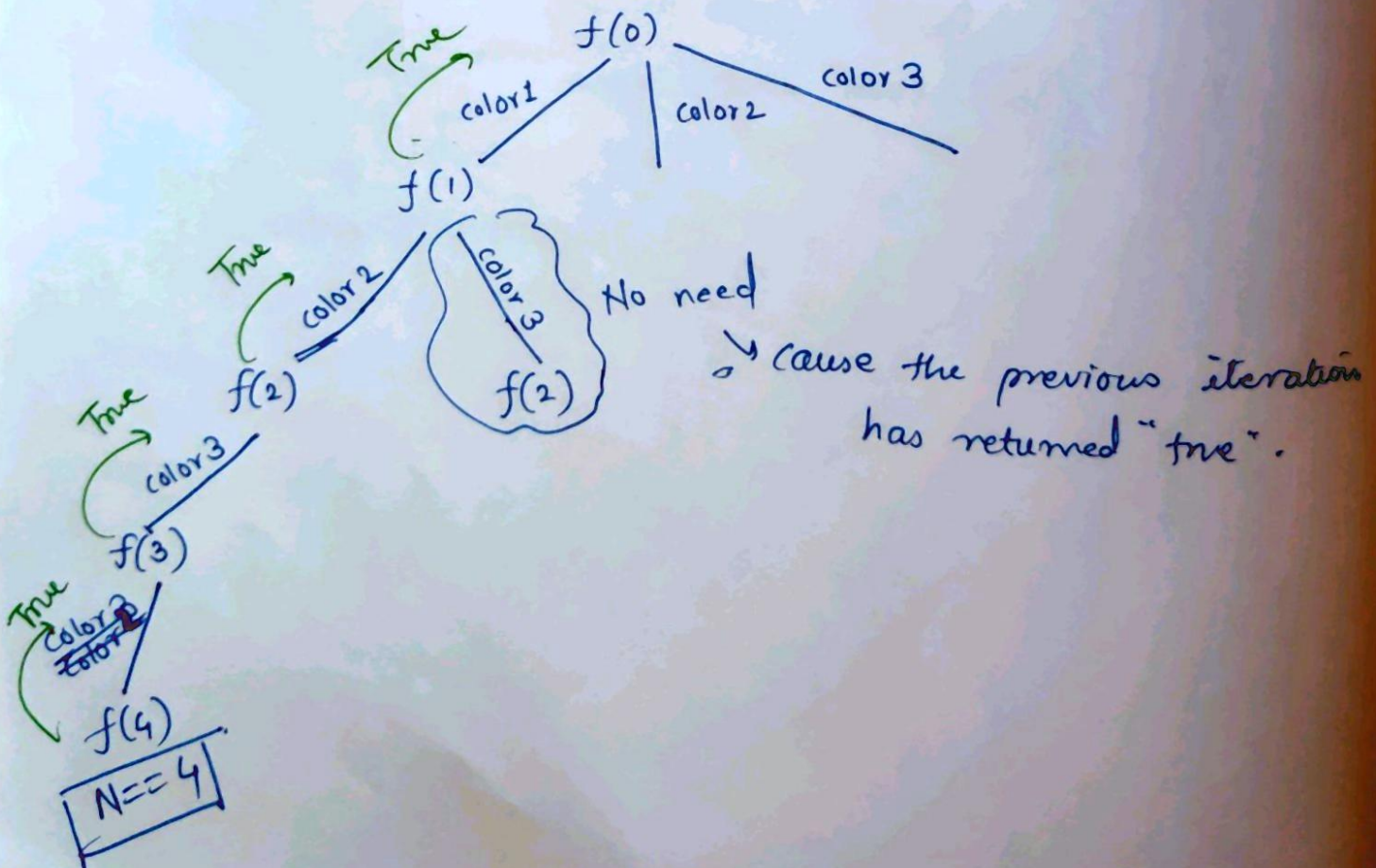
$N = 4$
 $M = 3$
 $E = 5$

Edges[] = $\{(1,2), (2,3), (3,4), (4,1), (1,3)\}$
 o/p = 1

None of the adjacent nodes have the same color Hence we can color the graph.



$M = 3$
 $\downarrow$
 No. of Colors



pseudo code :

f(node) {

if (node == N) $\longrightarrow$ Base case
return true

for (color = 1 $\longrightarrow$ M) {

if (possibleColor(node)) ✓ {

color[node] = col ;

if (f(node + 1) == true) $\longrightarrow$ if it giving $\Rightarrow$ T
return true; then we can return true

color[node] = 0 $\longrightarrow$ Remove the color
}

}

return false; $\longrightarrow$ It's not possible to color the graph.

}

● Palindrome Partitioning :-

Given a string s , partition ' s ' such that every substring of the partition is a palindrome. Return all possible palindrome partition of string ' s '.

Ex:- $s = "aab"$
output = $[["a", "a", "b"], ["aa", "b"]]$

Ex:- ① $s = "aabb"$

→ if $a|a|b|b$ → then partition $\{ "a", "a", "b", "b" \}$
partitions

$a|a|bb$ → $\{ "a", "a", "bb" \}$

$aa|b|b$ → $\{ "aa", "b", "b" \}$

$aab|b$ → $\{ "aab", "b" \}$

Output:-

$\{ \{ "a", "a", "b", "b" \},$

$\{ "a", "a", "bb" \},$

$\{ "aa", "b", "b" \},$

$\{ "aab", "b" \} \}$

⇒ as $1 \leq s.length() \leq 16$

so we can apply brute force.

Code :-

```
vector<vector<string>> partition(string s) {  
    vector<vector<string>> res;  
    vector<string> path;  
    func(0, s, path, res);  
    return res;  
}  
  
void func(int index, string s, vector<string> &path, vector<vector<string>> &res) {  
    if (index == s.size) {  
        res.push_back(path);  
        return;  
    }  
  
    for (int i = index; i < s.size; i++) {  
        if (isPalindrome(s, index, i)) {  
            path.push_back(s.substr(index, i - index + 1));  
            func(i + 1, s, path, res);  
            path.pop_back();  
        }  
    }  
}  
  
bool isPalindrome(string s, int start, int end) {  
    while (start <= end) {  
        if (s[start] != s[end]) {  
            return false;  
        }  
        start++;  
        end--;  
    }  
    return true;  
}
```


• Rat in a maze :-

Consider a rat placed at $(0,0)$ in a square matrix of order $n \times n$. It has to reach the destination at $(n-1, n-1)$. Find all possible paths that the rat can take to reach from source to destination. The directions in which the rat can move are 'U' (Up), 'D' (Down), 'L' (Left), and 'R' (Right). Value 0 cell in the matrix represents that it is blocked and rat cannot move to it while value 1 at cell in the matrix represents that rat can be travel through it.

Input :-

$n = 4$

matrix[][] = { {1, 0, 0, 0},
 {1, 1, 0, 1},
 {1, 1, 0, 0},
 {0, 1, 1, 1} }

output $\Rightarrow$

DDRDRR DRDDRR

1	0	0	0
↓	↓	↓	↓
1	→	0	1
↓	↓	↓	↓
1	→	1	0
0	↓	↓	↓
0	1	→	→
	↓	→	→

DRDDRR
DDRDRR

Lexicographically

DDRDRR < DRDDRR

Source

0	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1

$f(\text{start, end, ...})$
 $f(0, 0, \text{""})$

Lexicographically order of moves
 D L R U

0	0	1	2	3
0	✓			
1	✓			
2	✓	✓		
3		✓	✓	✓

visited

$f(0, 0, \text{""})$ (D, L, R, U)

$f(1, 0, \text{"D"})$ (D, L, R, U)

you cannot go upwards
 as upward cell is visited

$f(2, 0, \text{"DD"})$ (D, L, R, U)

$f(2, 1, \text{"DDR"})$ (D, L, R, U)

get back

$f(1, 1, \text{"DDR U"})$

$f(3, 1, \text{"DDRD"})$ (D, L, R, U) (Already visited)

$f(3, 2, \text{"DDRDR"})$ (D, L, R, U) (Already visited)

$f(3, 3, \text{"DDRDRR"})$

Now after returning, we have to
 mark (3, 3) as unvisited → but we
 can't take any move



! We have reached
 to our
 destination!!

$f(1, 0, "DR")$

Right move

$f(1, 1, "DR")$ $\checkmark$ O L R U

$f(2, 1, "DRD")$ $\checkmark$ D L R U

$f(3, 1, "DRDD")$ $\checkmark$ D L $\checkmark$ R U

$f(3, 2, "DRDDR")$ $\checkmark$ D L $\checkmark$ R U

$f(3, 3, "DRDDRR");$

Reached!

Pseudo Code

$f()$ {

Down

$f();$

Left

$f();$

Right

$f();$

Up

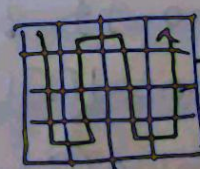
$f();$

}

Time complexity = $O(4^{m \times n})$

Space Complexity = Depth of the maximum recursive tree

$O(m \times n)$



→ All possibilities


```

vector<string> findPath(vector<vector<int>> &a, int n) {
    vector<string> ans;
    vector<vector<int>> visited(n, vector<int>(n, 0));
    if (a[0][0] == 1)
        solve(0, 0, a, n, ans, "", vis);
    return ans;
}

```

```

void solve(int i, int j, vector<vector<int>> &a, int n, vector<string> &ans, string move,
          vector<vector<int>> &visited)
{
    if (i == n-1 && j == n-1)
        return ans.push_back(move);
        return;

```

// Downward direction

```

    if (i+1 < n && !visited[i+1][j] && a[i+1][j] == 1) {
        visited[i][j] = 1;
        solve(i+1, j, a, n, ans, move + 'D', visited);
        visited[i][j] = 0;
    }

```

// Left direction

```

    if (j-1 >= 0 && !visited[i][j-1] && a[i][j-1] == 1) {
        visited[i][j] = 1;
        solve(i+1, j, a, n,
        solve(i, j-1, a, n, ans, move + 'L', visited);
        visited[i][j] = 0;
    }

```


// Right direction

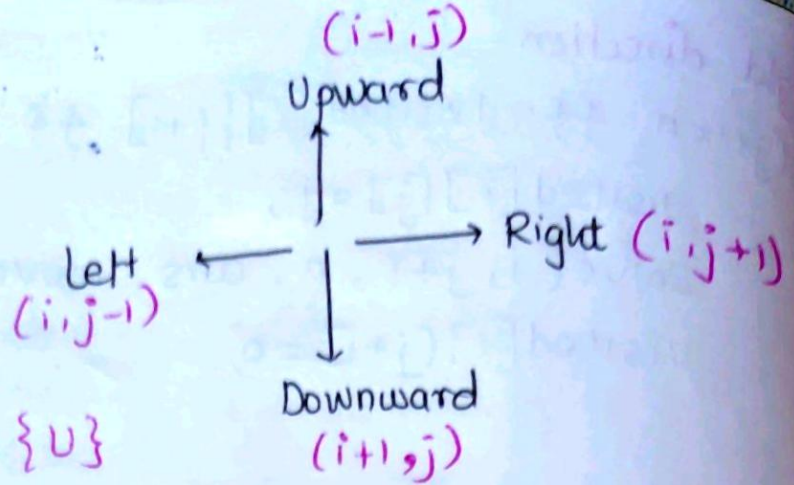
```
if (j+1 < n && !visited[i][j+1] && a[i][j+1] == 1){  
    visited[i][j] = 1;  
    solve(i, j+1, n, ans, move + 'R', visited);  
    visited[i][j+1] = 0;  
}
```

// upward direction

```
if (i-1 >= 0 && !visited[i-1][j] && a[i-1][j] == 1){  
    visited[i][j] = 1;  
    solve(i-1, j, n, ans, move + 'U', visited);  
    visited[i-1][j] = 0;  
}
```

→ This is a bit lengthy code:

→ We can shorten this code by using for loop.



$\{D\}$ $\{L\}$ $\{R\}$ $\{U\}$

$di[] = \{+1, +0, +0, -1\}$

$dj[] = \{+0, -1, +1, +0\}$

Current index gets $\rightarrow +1$
and 'j' gets $\rightarrow +0$

So we can do

$i + di[index]$
 $j + dj[index]$

Because of this you
don't need to write 4 different
if statements

```
void solve(int i, int j, vector<vector<int>> &a, int n, vector<string> &ans,
string move, vector<vector<int>> &visited, int di[], int dj[]) {
    if(i == n-1 && j == n-1) {
        ans.push-back(move);
        return;
    }
    string direction = "DLRU";
    for(int index = 0; index < 4; index++) {
        int nexti = i + di[index];
        int nextj = j + dj[index];
        if(nexti >= 0 && nextj >= 0 && nexti < n && nextj < n
        && !visited[nexti][nextj] && a[nexti][nextj] == 1) {
            visited[i][j] = 1;
            solve(nexti, nextj, a, n, ans, move + direction[index],
            visited, di, dj);
            visited[i][j] = 0;
        }
    }
}
```



```

vector<string> findPath(vector<vector<int>> &m, int n, int m, int n) {
    vector<string> ans;
    vector<vector<int>> visited(vector<int>(n, 0));
    int di[] = { 1, 0, 0, -1 };
    int dj[] = { 0, -1, 1, 0 };
    if (m[0][0] == 1)
        solve(0, 0, m, n, ans, "", visited, di, dj);
    return ans;
}

```


• k^{th} permutation Sequence

The set $[1, 2, 3, \dots, n]$ contains a total of " $n!$ " unique permutations.

By listing and labeling all of the permutations in order, we get the following sequence for $n=3$.

① "123"

② "132"

③ "213"

④ "231"

⑤ "312"

⑥ "321"

Given ' n ' & ' k ' find the k^{th} permutation sequence.

Example 1 :-

Input :- $n=3, k=3$

output :- "213"

① Brute Force :-

- ① Generate all the permutations by using recursion.
- ② Store all the permutation into a data structure.
- ③ Sort the data structure.
- ④ Return the $(k-1)^{\text{th}}$ ~~per~~ element from the ds.

Time Complexity :- $O(n! \times n)$

To generate all permutation

Store the permutation into a ds

② Optimal Solution :-

Example :- $n=4$, $k=17$

#. of permutations = $n!$

$$= 4!$$

$$(4!) = 24$$

$[1, 2, 3, 4]$

indexing $k=17 \rightarrow 16^{\text{th}}$ permutation

$(0 \rightarrow 5)$ pick 1 + $(2, 3, 4) \rightarrow 6$ permutations

$(6 \rightarrow 11)$ pick 2 + $(1, 3, 4) \rightarrow 6$ permutations

$(12 \rightarrow 17)$ In this range pick 3 + $(1, 2, 4) \rightarrow 6$ permutations

$(18 \rightarrow 23)$ pick 4 + $(1, 2, 3) \rightarrow 6$ permutations

1 2 3 4 $\rightarrow 0^{\text{th}}$ permutation (0-based indexing)

4 3 2 1 $\rightarrow 23^{\text{rd}}$ permutation

$k=17 \rightarrow 16^{\text{th}}$ permutation

Each no. has 6 permutation

So $16/6 = \boxed{2}$ index means

$\begin{matrix} 0 & 1 & \boxed{2} & 3 \\ \textcircled{1} & [1, 2, 3, 4] \end{matrix}$

It will start from this number.

3 _ _ _

$16 \div 4 = 4^{\text{th}}$ sequence out of

$$3 + (1, 2, 4)$$

this sequence

then our job will be done.

$[1, 2, 4] \rightarrow$ Remaining numbers
and we want 4th permutation

So Now question is

$$[1, 2, 4] \quad k=4$$

Again Same process

(0-1) pick $\rightarrow 1 + \{2, 4\} \rightarrow 2$ permutations

(2-3) pick $\rightarrow 2 + \{1, 4\} \rightarrow 2$ permutations

(4-5) pick $\rightarrow 4 + \{1, 2\} \rightarrow 2$ permutations

kth permutation $\rightarrow 4/2 = 2$ $\begin{matrix} 0 & 1 & 2 \\ [1, & 2, & 4] \end{matrix}$

permutation is going to start with 4

3 4 _ _

$4 \div 2 = 0 \rightarrow$ Now question boils down to

$$[1, 2] \quad k=0$$

Again Same process

$$[1, 2] \quad k=0$$

(0-0) pick $\rightarrow 1 + \{2\} \rightarrow 1$ permutations

(1-1) pick $\rightarrow 2 + \{1\} \rightarrow 1$ permutation

$$k^{\text{th}} \text{ permutation} = 0/1 = \underline{\underline{0}} \quad [1, 2]$$

means permutation is going to start with

1.

~~3~~
3 4 1 _

$\downarrow$

$$k = 0/1 = 0 \rightarrow [2] \quad k=0$$

Of again same process

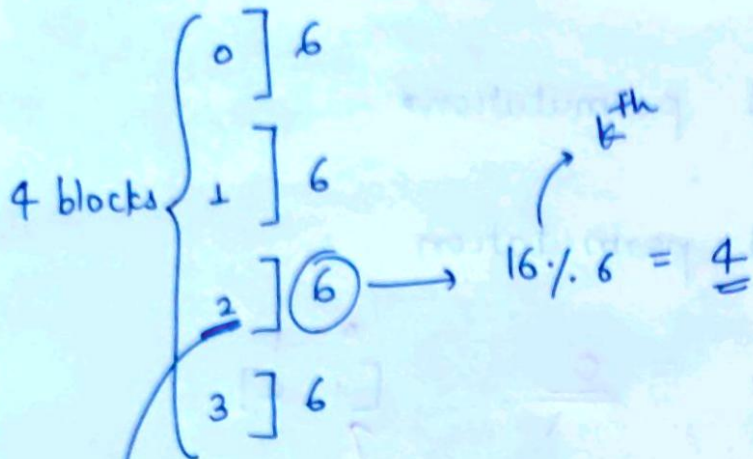
we will get

3 4 1 2

$\Rightarrow$ This is our answer!!

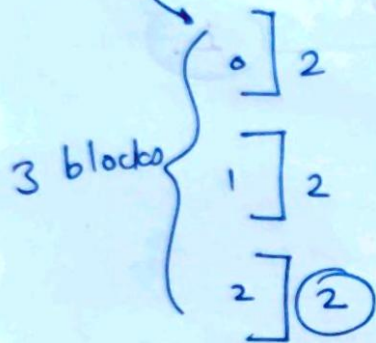
Summary :-

At first,



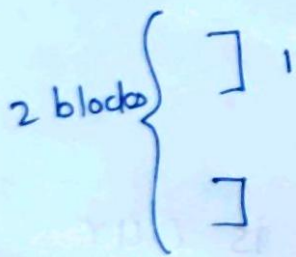
$$6 + 6 + 6 + 6 = 24$$

$$4! = 24$$



$$2 + 2 + 2 = 8$$

$$3! = 8$$



$$1 + 1 = 2$$

$$2! = 2$$

In each pass, we are taking/picking 1 element out till (n) so $O(N)$

like this

Time Complexity : $O(N) * O(N) \Rightarrow O(N^2)$

$n=4 \rightarrow$ we are looking for 4 numbers

so $O(N)$ will take

Space Complexity = $O(N)$

Code

WHY?

because initially we have ~~made~~ ~~to~~ do the division in terms of blocks if $n=4 \rightarrow 24$ permutations but we had 4 blocks of 6 permutation

This is the factorial.

Hence (n) is taken for $n=4 \rightarrow$ we have to compute $3!$

```
string getPermutation (int n, int k){
```

```
int fact = 1;
```

```
vector<int> numbers;
```

```
for(int i=1; i<=n; i++){
```

```
    fact = fact * i;
```

```
    numbers.push_back(i);
```

$\rightarrow$ This will store $[1, 2, 3]$

```
}
```

```
numbers.push_back(n);
```

$\rightarrow$ This will add 'n' to last position $[1, 2, 3, 4]$

```
string ans = "";
```

```
k = k - 1;
```

$\rightarrow$ (0-based) indexing hence we have reduced a value by 1.

```
while (true) {
```

```
    ans += to_string(numbers[k/fact]);
```

$k=16$

```
    numbers.erase(numbers.begin() + (k/fact));
```

Initially we had 4 blocks of 6.

$16/6 = 2$ we have done

& we got '2' and then we have taken the 2nd number

$[1, 2, 3, 4]$

Taken ✓

Add it to ans.

```
    if(numbers.size() == 0) break;
```

```
    k = k % fact
```

After adding, our next 'k' value will be $16 \% 6 = 4$

```
    fact = fact / numbers.size();
```

Now, block size will be '2' Hence we have to divide $6/3 = 2$

```
}
```

```
return ans;
```